

How it's built

A map of SecureGate's files. You don't need to read code to change its behavior: the last table says which file to edit.

Folders

<code>policy.yaml</code>	the rules that decide block / warn / ignore
<code>.gitleaks.toml</code>	which key shapes Gitleaks looks for
<code>.pre-commit-config.yaml</code>	the laptop gate (installed with make hooks)
<code>.github/workflows/secret-gate.yml</code>	the merge gate: the check on every pull request
<code>Makefile</code>	short commands: make setup, make test, make demo, ...
<code>src/securegate/</code>	the program
<code>tools/</code>	helpers for the gates and the docs
<code>tests/</code>	automatic checks that prove the program works
<code>docs/</code>	these pages

Modules

A **module** is one Python file with one job. When you run a scan, they work in this order:

Module	Job
<code>cli.py</code>	Reads your command (<code>securegate scan ...</code>), runs it, and returns the exit code.
<code>scanners/gitleaks.py</code>	Runs Gitleaks, reads its report from a temporary folder, then deletes the folder.
<code>pipeline.py</code>	The only place raw secrets pass through. It turns each one into a masked finding.
<code>entropy.py</code>	Measures how random a value looks.
<code>policy.py</code>	Reads <code>policy.yaml</code> and decides block, warn or ignore.
<code>confidence.py</code>	Scores from 0 to 1 how sure we are that a finding is a real secret.
<code>mask.py</code>	Hides the value and makes its fingerprint.
<code>finding.py</code>	The finding record. It can only hold a masked value.
<code>report.py</code>	Prints the table, the "why and fix" lines for blocked findings, and writes <code>findings.json</code> .
<code>summary.py</code>	Turns <code>findings.json</code> into a short Markdown summary (<code>securegate summary</code>), used on GitHub's check page.
<code>demo/</code>	Builds the demo repo: <code>catalog.yaml</code> (what to plant), <code>generator.py</code> , <code>scorecard.py</code> .
<code>ui/</code>	The read-only dashboard: <code>app.py</code> (the pages), <code>report_view.py</code> (reads and checks <code>findings.json</code>), <code>fixes.py</code> ("How to fix"), <code>server.py</code> (127.0.0.1 only), <code>templates/</code> and <code>static/css/</code> .
<code>errors.py</code> , <code>validate.py</code> , <code>programs.py</code>	Helpers: error types, checks for hand-edited files, finding programs safely.

The Finding format

Each entry in `findings.json` is one finding. This real example comes from scanning the demo repo:

Field	Example	Meaning
id	dcc7b5bbfe3c	Short name: the first 12 characters of the fingerprint. The same secret always gets the same id.
rule	stripe-access-token	The Gitleaks rule that matched.
detector	gitleaks	The scanner that found it.
file	scripts/migrate_customers.py	Where, relative to the scanned folder.
line	3	The line where the value starts.
commit	720d024c...	The commit that added it. Empty (null) when only files on disk were scanned.
author	Riya Demo	Who made that commit.
date	2025-06-04T09:00:00Z	When.
masked_value	sk_1****562d	The value with all but 4 + 4 characters hidden.
fingerprint	dcc7b5bb... (64 characters)	Recognizes the same secret again. It cannot be turned back into the secret.
entropy	4.625	How random the value looks (random keys: about 4 to 6).
confidence	0.9	0 to 1: how sure we are that it is a real secret.
severity	critical	critical, high, medium, low or info, from the policy.
decision	block	block, warn or ignore, from the policy.
reason	provider-keys: A payment, cloud or ...	Which policy rule decided, and why.
remediation	Treat it as leaked. Rotate ...	What to do next.

The file also has a summary at the top: status (pass, fail or error), exit code, Gitleaks version and counts per decision. After an error it has no findings list at all, so nobody can mistake a failed scan for a clean one.

Exit codes

Code	Meaning
0	Pass. Nothing to block (there may be warnings).
1	At least one finding is "block".
2	SecureGate could not do its job: Gitleaks missing, a broken <code>policy.yaml</code> , an unreadable report. It never says "pass" when something went wrong.

Where to change what

I want to...	Change this
block, warn or ignore a kind of finding	<code>policy.yaml</code> : add or move a rule
treat a folder differently	<code>policy.yaml</code> : <code>path_matches</code>
ignore a known fake value	<code>policy.yaml</code> : <code>value_matches</code> in the <code>placeholders</code> rule
change the "why" or "fix" line shown for a blocked finding	<code>policy.yaml</code> : <code>reason</code> and <code>remediation</code> of that rule
detect a new kind of key	<code>.gitleaks.toml</code> : add a <code>[[rules]]</code> entry, like <code>acme-pay-token</code>

I want to...	Change this
plant a new secret or decoy in the demo	<code>src/securegate/demo/catalog.yaml</code> (see Testing (testing.pdf))
change how values are masked	<code>src/securegate/mask.py</code>
change the table or <code>findings.json</code>	<code>src/securegate/report.py</code>
change the confidence score	<code>src/securegate/confidence.py</code>
change the dashboard's colours, fonts or spacing	<code>src/securegate/ui/static/css/tokens.css</code> (see Dashboard (ui.pdf))
change what a dashboard page shows	<code>src/securegate/ui/templates/</code>
change the "How to fix" advice	<code>src/securegate/ui/fixes.py</code>
change what the check on pull requests does	<code>.github/workflows/secret-gate.yml</code> (see The two gates (merge-gate.pdf))
change the check before each commit	<code>.pre-commit-config.yaml</code> and <code>tools/precommit_hook.py</code>
update the PDF copies of these pages	edit the <code>.md</code> page, then run <code>make docs-pdf</code> (<code>tools/docs_pdf.py</code>)

When you change a behavior, update the matching page in `docs/` in the same commit, and run `make docs-pdf` so its PDF matches. A test fails when a PDF is out of date.